

LTIM_AIServicesB2_MS2Set1_Q1

Test Summary

No. of Sections: 1
No. of Questions: 1
Total Duration: 900 min

Section 1 - COD

Additional Instructions:

None

Q1. COLD STORAGE ELECTRICITY CONSUMPTION FORECAST

PROBLEM STATEMENT

A temperature-controlled warehouse runs refrigerated chambers around the clock and pays a demand-linked electricity tariff. Its operations desk has to know what a storage run will consume before a chamber is assigned, not after the energy meter is read.

Consumption is not a simple multiple of any one setting. A heavier stored load increases the refrigeration work; the gap between outside temperature and the chamber setpoint raises compressor demand; time spent with the doors open admits warm air; older compressors draw more power; and the storage mode, insulation grade and rooftop solar supply alter the final draw. A forecasting stage is required to learn that shape from metered runs, to confirm that the model chosen for the job is the better of the two considered, and to forecast electricity for runs still queued.

OBJECTIVE

Build a six-stage pipeline that prepares the operations exports for modelling, fits two forecasters and stores each of them, ranks the two against runs held back from fitting, measures the chosen forecaster on those held-back runs, and applies the stored forecaster to a queue of runs whose electricity has not yet been metered.

FILE STRUCTURE

```
Project
|-- data
|   |-- cold_storage_train.csv
|   |-- cold_storage_test.csv
|   `-- cold_storage_predict.csv
|-- processed_data
|-- artifacts
|-- output
|-- main.py
`-- requirements.txt
```

PURPOSE OF FILES

Path	Purpose
data/cold_storage_train.csv	Metered runs used to fit the forecasters.
data/cold_storage_test.csv	Metered runs held back from fitting, used to rank and measure.
data/cold_storage_predict.csv	Queued runs with no meter reading yet.
processed_data	Destination for the model-ready tables.
artifacts	Destination for the two fitted forecasters.
output	Destination for the issued forecasts.
main.py	Module to be implemented.
requirements.txt	Package versions required by the pipeline.

DATASET DESCRIPTION

Training Dataset: cold_storage_train.csv

Column	Role	Description
run_id	Record	Reference for the storage run, for example CS-2026-000241.
operation_date	Record	Date the chamber run was recorded.
facility_location	Record	Cold-storage facility that ran it.
chamber_code	Record	Refrigerated chamber used.
shift_supervisor	Record	Supervisor on the floor at the time.
client_contract_ref	Record	Customer contract reference.
stored_load_tonnes	Feature	Stored product load in tonnes, from 8 to 120.
outside_temperature_celsius	Feature	Ambient temperature around the facility, from 18 to 46.
door_open_minutes	Feature	Total chamber door-open time during the run, from 5 to 180 minutes.
compressor_age_months	Feature	Age of the refrigeration compressor, from 3 to 144 months.
temperature_setpoint_celsius	Feature	Chamber setpoint, from -24 to 8 degrees Celsius.
storage_mode	Feature	Operating mode: frozen, chilled, controlled or mixed.
insulation_grade	Feature	Thermal envelope: standard, enhanced or high-efficiency.
uses_rooftop_solar	Feature	Whether rooftop solar supplied the facility during the run: no or yes.
electricity_consumed_kwh	Target	Electricity metered for the run in kWh.

SAMPLE RECORDS

Column	Record 1	Record 2	Record 3
run_id	CS-2026-000241	CS-2026-004822	CS-2026-011507
operation_date	2026-03-18	2026-05-07	2026-06-12
facility_location	Nashik	Pune	Nagpur
chamber_code	CH-014	CH-038	CH-061
shift_supervisor	S-Iyer	P-Patil	N-Rao
client_contract_ref	CC-55102	CC-82714	CC-30491
stored_load_tonnes	34.8	91.5	57.2
outside_temperature_celsius	31.4	(blank)	42.1
door_open_minutes	48.0	122.5	76.2
compressor_age_months	18	73	(blank)
temperature_setpoint_celsius	-18.0	4.0	-6.0
storage_mode	FROZEN	chilled	MIXED
insulation_grade	standard	ENHANCED	high-efficiency
uses_rooftop_solar	No	YES	no
electricity_consumed_kwh	7278.45	13241.62	10988.30

cold_storage_test.csv holds rows of exactly this shape. cold_storage_predict.csv holds rows of the same shape with the final line, electricity_consumed_kwh, absent.

TASK

Implement the following functions and the main block described below inside main.py.

FUNCTION SIGNATURE

```
clean_data(input_path, output_path)
```

PARAMETERS

- input_path - string locating an operations export inside data.
- output_path - string locating the model-ready table to be written in processed_data.

REQUIREMENTS

- Drop the six record columns. Keep the eight feature columns and electricity_consumed_kwh where present.
- Remove repeated records, keeping the first occurrence. This must happen before any figure is measured from a column.
- Fill blanks in outside_temperature_celsius, door_open_minutes and compressor_age_months with the median of their own column, not the mean.
- Normalise case and stray spaces in the text columns before encoding.
- Encode storage_mode as frozen 0, chilled 1, controlled 2, mixed 3.
- Encode insulation_grade as standard 0, enhanced 1, high-efficiency 2.
- Encode uses_rooftop_solar as no 0, yes 1.
- Leave no text column: every surviving column, including electricity_consumed_kwh where present, must be numeric. Do not create electricity_consumed_kwh on an export that lacks it.
- Write to output_path with no index column.

RETURNS

pandas.DataFrame - the cleaned table.

FUNCTION SIGNATURE

```
train_tree_model(processed_path, model_path)
```

PARAMETERS

- processed_path - string locating the cleaned training table in processed_data.
- model_path - string locating the fitted forecaster to be stored in artifacts.

REQUIREMENTS

- Fit a DecisionTreeRegressor with max_depth=10 and random_state=42.
- Fit on every row of the file named by processed_path.
- Separate electricity_consumed_kwh from the columns fitted on.
- Persist the fitted forecaster to model_path so it reloads without refitting.

RETURNS

DecisionTreeRegressor - the fitted forecaster.

FUNCTION SIGNATURE

```
train_boosting_model(processed_path, model_path)
```

PARAMETERS

- processed_path - string locating the cleaned training table in processed_data.
- model_path - string locating the fitted forecaster to be stored in artifacts.

REQUIREMENTS

- Fit a GradientBoostingRegressor with n_estimators=200, max_depth=3, learning_rate=0.1 and random_state=42, on every row of processed_path.
- Separate electricity_consumed_kwh from the columns fitted on.
- Boosting adds one shallow tree per round, each correcting what earlier rounds left over. The depth limit and learning rate are both required.
- Persist the fitted forecaster to model_path so it reloads without refitting.

RETURNS

GradientBoostingRegressor - the fitted forecaster.

FUNCTION SIGNATURE

```
compare_models(tree_model_path, boosting_model_path, features_test, target_test)
```

PARAMETERS

- `tree_model_path` - string locating the stored decision tree in artifacts.
- `boosting_model_path` - string locating the stored boosted model in artifacts.
- `features_test` - DataFrame of cleaned runs held back from fitting.
- `target_test` - electricity metered for each of them.

REQUIREMENTS

- Load both forecasters from the paths given. Do not refit either.
- Score each on the runs passed in, not on any file read from disk.
- Report RMSE, MAE and R2 score for each.
- RMSE must be reported in kWh, not kWh squared. Use `root_mean_squared_error` from `sklearn.metrics`.
- Return columns `model_name`, `rmse`, `mae`, `r2_score` - one row per forecaster, ordered by `r2_score`, strongest first.
- Name the models `DecisionTreeRegressor` and `GradientBoostingRegressor`.
- Nothing is written to disk by this stage.

RETURNS

`pandas.DataFrame` - the comparison table.

FUNCTION SIGNATURE

```
evaluate_model(model_path, features_test, target_test)
```

PARAMETERS

- `model_path` - string locating a stored forecaster in artifacts.
- `features_test` - DataFrame of cleaned runs.
- `target_test` - electricity metered for each of them.

REQUIREMENTS

- Load the forecaster from `model_path`. Do not refit it.
- Score it against the runs passed in.
- Report RMSE, MAE and R2 score, measured as in the comparison stage.

RETURNS

tuple of three float values, in the order listed above.

FUNCTION SIGNATURE

```
predict_new_data(model_path, input_path, processed_path, output_path)
```

PARAMETERS

- `model_path` - string locating a stored forecaster in artifacts.
- `input_path` - string locating the queued runs in data.
- `processed_path` - string locating the model-ready form of that queue.
- `output_path` - string locating the issued forecasts in output.

REQUIREMENTS

- Load the forecaster from `model_path`. Do not refit it.
- Prepare the queue at `input_path` with the same stage used for training, writing that prepared form to `processed_path`.
- `run_id` is removed during preparation, so read it from the queued export and keep it paired with the electricity forecast for that same run.
- Every run receives exactly one forecast.
- Write `run_id` and `predicted_electricity_consumed_kwh` only to `output_path`, with no index column.

RETURNS

`pandas.DataFrame` - the forecast queue.

MAIN BLOCK

- Runs the six stages in sequence and is the only place that prints.
- Prepares the training export and held-back export, then separates held-back runs into features and metered electricity.
- Fits and stores both forecasters, then ranks them.
- Reads the model named in the first row of the comparison table and passes that forecaster path into evaluation and prediction.
- Reports the shapes, storage confirmation, comparison table, three measures, and forecast queue shape.

GENERATED FILES

Path	Contents
processed_data/cold_storage_train_cleaned.csv	Model-ready training runs with their meter readings.
processed_data/cold_storage_test_cleaned.csv	Model-ready held-back runs with their meter readings.
processed_data/cold_storage_predict_cleaned.csv	Model-ready queued runs.
artifacts/decision_tree_model.pkl	Fitted decision tree.
artifacts/gradient_boosting_model.pkl	Fitted boosted model.
output/cold_storage_energy_predictions.csv	Electricity forecast for each queued run.

EXPECTED OUTPUT

```
Cleaned training runs: (27000, 9)
Cleaned held-back runs: (7715, 9)
Models trained and saved.
Model comparison:
      model_name      rmse      mae  r2_score
GradientBoostingRegressor 494.685254 383.697348 0.927353
      DecisionTreeRegressor 831.460643 660.543594 0.794770
RMSE: 494.69 | MAE: 383.70 | R2: 0.9274
Predictions written: (800, 2)
```

The row counts fall below the counts in the exports because repeated records are removed. The figures vary only slightly between runs.

IMPLEMENTATION STEPS

STEP 1 - Move into the project directory.

```
cd Project
```

STEP 2 - Install the pinned packages.

```
pip install -r requirements.txt
```

STEP 3 - Run the pipeline.

```
python3 main.py
```

STEP 4 - To run the testcases, click the Run Testcases button or run:

```
python3 -m pytest ../pytest/tests.py -v
```

Answer Key & Solution

Section 1 - COD

Q1 Solution cannot be displayed